HANDS-ON

Kiro ハンズオン

はじめての AI駆動開発を、ひとりで一歩ずつ

github.com/multilayer/kiro-handson-practice

このハンズオンの前提



Kiro は自分で用意

GitHub などでサインイン。
無料プランから始められます



自分の GitHub にコピー

「Use this template」で
自分のリポジトリを作り、
その中で自由に練習します



ひとりで進められる

レビュー役も依頼者役も
Kiro が担当。
仲間との勉強会にも使えます

全体の地図：準備＋全10回

STEP 0
始める前の準備

STEP 1
ログインとルール

STEP 2
GitHub とクローン

STEP 3
チャットに慣れる

STEP 4
渡し方とやり直し方

STEP 5
Steering と Hooks

STEP 6
Spec で要件を作る

STEP 7
Spec でツールを作る

STEP 8
モブ演習

STEP 9 応用
スキルと MCP

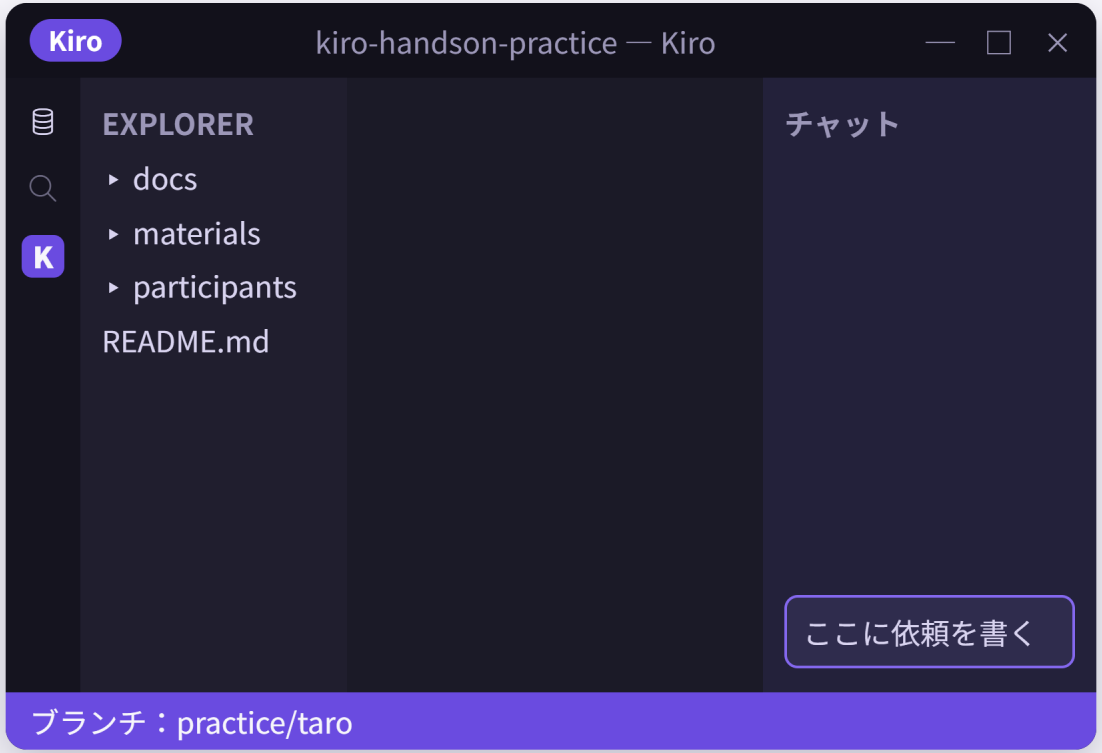
STEP 10 上級
AI-DLC

準備　基本　Spec（仕様駆動開発）　応用・上級

1回 30〜90分。前の回で作ったものを次の回で使うので、STEP 0 から順に進めます

進め方：手順書を左、Kiro を右に並べる





スライドで全体をつかむ

操作は手順書（docs/）を見ながら

時間は目安の1.5倍を見ておく

STEP 0 始める前にそろえる5つ

作業フォルダ
kiro フォルダを
作っておく

Git
git-scm.com から
入れる（無料）

GitHub
アカウントを作る
（無料）

Kiro IDE
kiro.dev から入れる

リポジトリ
自分のアカウントに
コピー

Git が入ったか確かめる （PowerShell）

PowerShell Windows PowerShell

```
PS> git --version
git version 2.xx.x
```

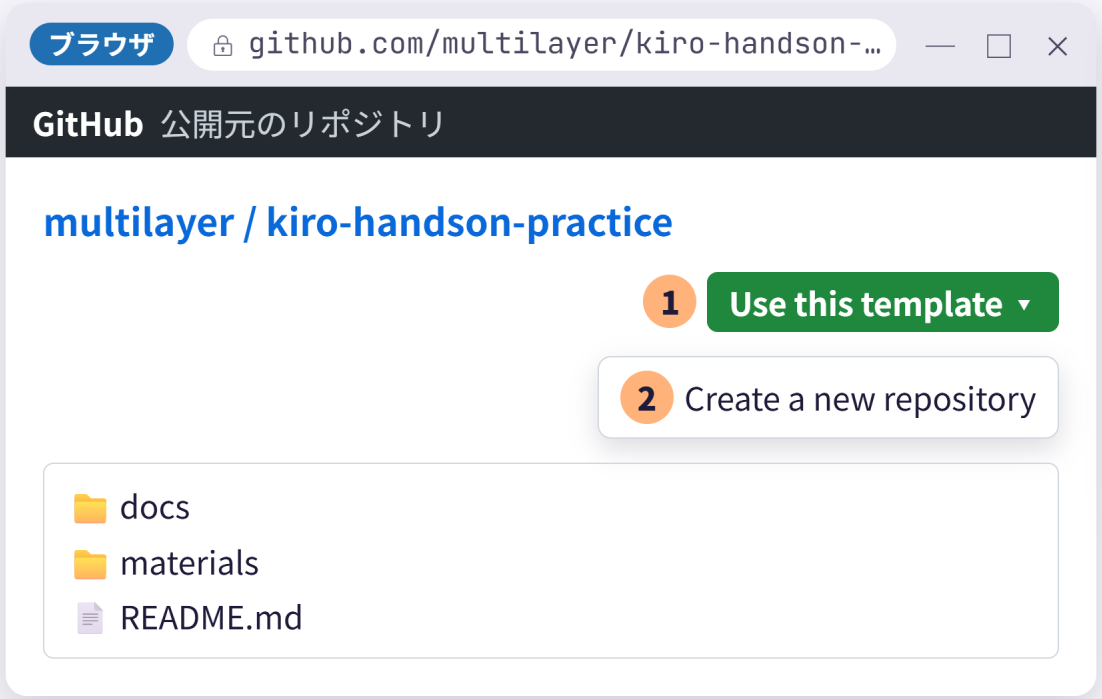
STEP 0 Kiro のプランとクレジット

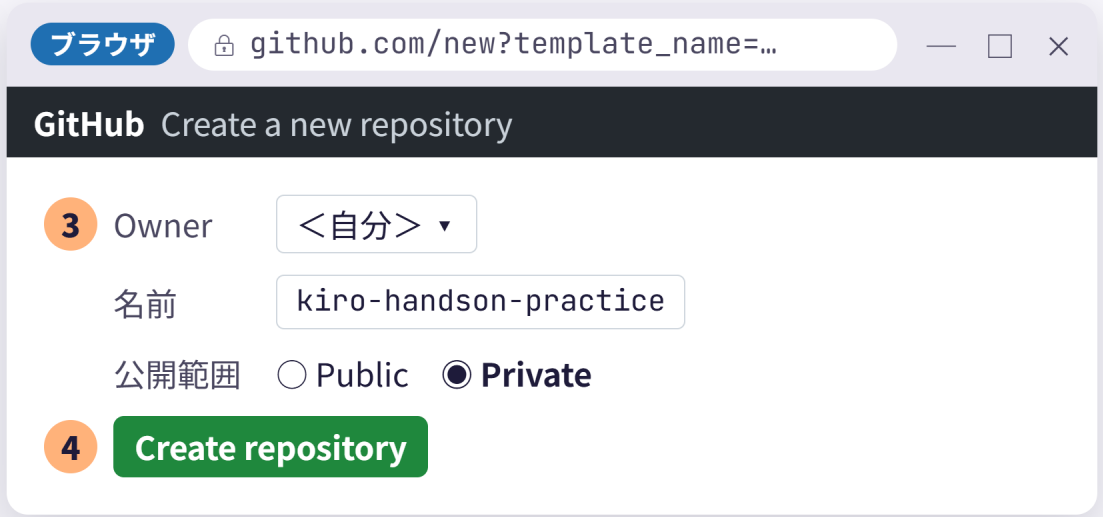
Free	Pro　おすすめ	Pro+ 以上
50 /月	**1,000 /月**	**2,000〜**
無料。STEP 1〜3 のお試し向き	月20ドル。 STEP 1〜9 を通して行うならこれ	月40ドル〜。 STEP 10 を何度も試したい人向け

- STEP 10（AI-DLC）は上位モデル（Claude Opus）を使うため **有料プランが必要**
- モデルは手順書の指示がない限り **Auto のまま**。高性能なモデルほどクレジットが早く減ります

2026年10月時点の kiro.dev/pricing の内容です。申し込む前に最新を確認してください

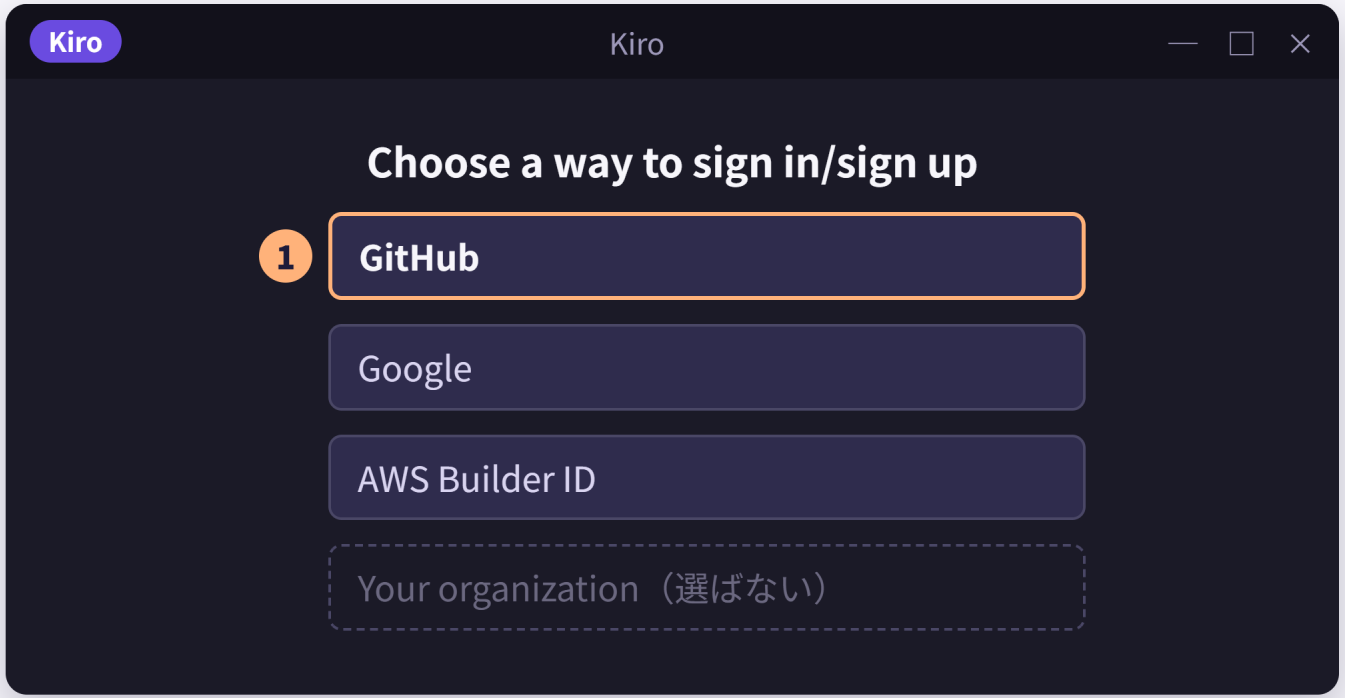
STEP 0 ブラウザ 自分の GitHub にコピーする





自分のリポジトリ github.com/＜自分＞/kiro-handson-practice ができます。
以降の手順は、すべてこのリポジトリで進めます

STEP 1 Kiro Kiro にサインインする



1. 使う方法を選ぶ。GitHub がおすすめ
2. ブラウザが開くのでサインインし、許可する
3. Kiro に戻ったら、チャット欄を開く

   `Ctrl` + `L`

初めての人は、ここで無料プランのアカウントができます

STEP 1

必ず守る4つのルール

個人情報・顧客情報は入れない
練習には用意されたサンプル資料だけを使う

秘密情報も入れない
パスワード・API キー・トークン

迷ったら、入れない・許可しない
止まって Kiro に「これは何？」と聞く

main は直接変えない
自分用のブランチで作業する

入れてはいけない例：自分や他人の氏名・電話番号・住所・口座番号、パスワード、API キー
エラー画面を貼るときも、写り込んでいないか確かめる

STEP 2 GitHub の言葉は4つだけ

リポジトリ
変更履歴が
全部残るフォルダ

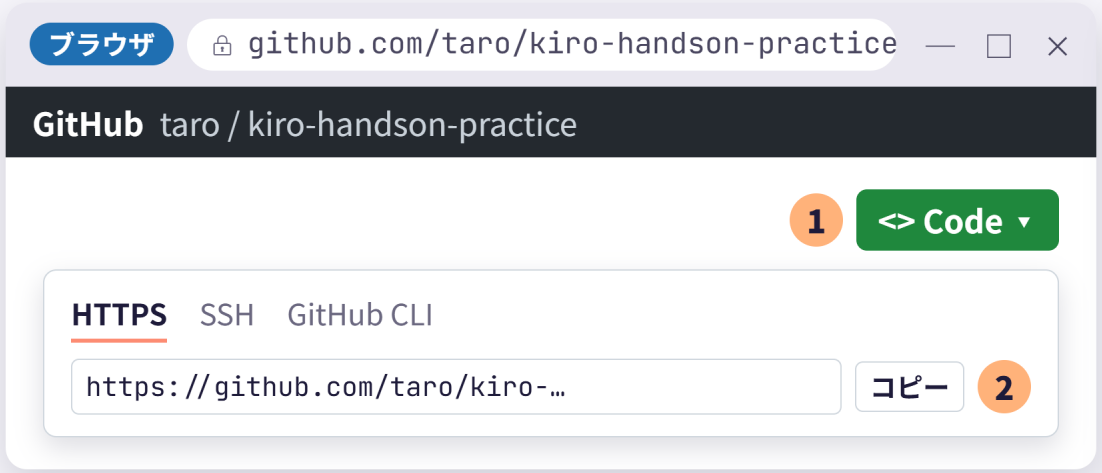
コミット
保存。
いつ・誰が・何を・なぜ

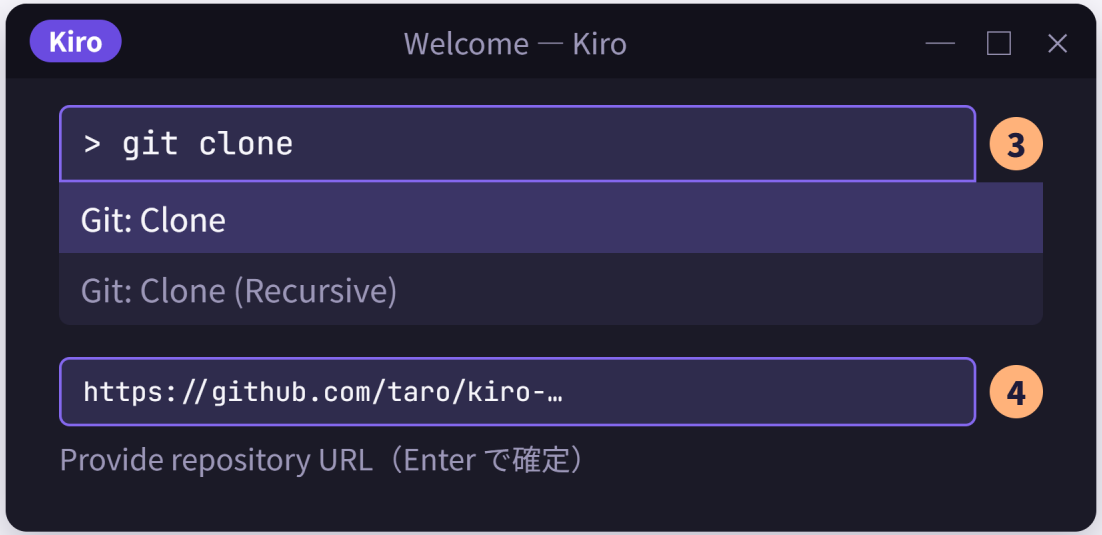
ブランチ
作業用のコピー。
main を汚さない

プルリクエスト
本体に取り込む前の
見直し依頼



STEP 2 自分のリポジトリを PC に取り込む（クローン）





1. 自分のリポジトリで Code を押す
2. HTTPS の URL をコピー
3. Kiro で Ctrl＋Shift＋P → Git: Clone
4. URL を貼って Enter → 保存先を選ぶ C:\Users\＜自分＞\kiro

> ⚠ URL は **＜自分のユーザー名＞/kiro-handson-practice**。
> 初回は GitHub へのサインインを求められるので Authorize。開いたら Open → Trust を選ぶ

STEP 2 Kiro ブランチは Kiro に頼んで作る



チャットに送る（taro は自分の名前に）

1 許可を求められたら、読んでから Run

自分のブランチ名が入っているか

main を消す・上書きする → Reject して止まる

分からない →「これは何をするもの？」と聞く

2 左下が practice/taro に変われば完了

main のままなら、もう一度頼む

STEP 3 良い頼み方の型は4つ

① 役割

② 資料（#ファイル名）

③ ほしい形

④ 守ってほしいこと

Kiro のチャットに入力

① あなたは業務部門とベンダーの間を取りまとめる担当者のアシスタントです。

② **#materials/議事録_サンプル_変更依頼定例.md** を読んで、

③ 決まったこと／持ち越し／ToDo（担当・期限の表）に整理してください。

④ 推測で補わず、議事録に書いてあることだけを使ってください。

演習：要約 → 頼み方を比べる → **抜けを質問させる** → 食い違いを探す

資料には、わざと食い違いを3か所入れてある

STEP 4 Kiro 資料の渡し方と、やり直し方



1 入力欄で # → 渡すものを選ぶ

File＝ファイル1つ
Folder＝フォルダごと
Codebase＝全体から探す

2 失敗したら依頼の上の Restore

ファイルも会話もその前に戻る

コミット・プッシュ済みは戻らない

3 監督モード（Supervised）

Kiro が直すたびに止まる。
赤＝消える行、緑＝増える行を見てから Accept

STEP 4〜 保存と送信も Kiro に頼む



Kiro のチャットに入力

participants/taro/ の変更を「ベンダー依頼文の清書を追加」というメッセージでコミットして、
GitHub にプッシュしてください。

許可の前に確認：自分のフォルダ名・ブランチ名が入っているか。**main に直接送るコマンドなら Reject する**

STEP 5 Steering で前提を伝え、Hooks で自動チェック

Steering

Kiro が毎回読む、業務の説明書

```
.kiro/steering/rules.md
```

自分用のメモは **manual** にして、/memo-taro と
呼んだときだけ使う

Hooks

決まったタイミングで、決まった作業を自動で行う

.md を
保存したら → 個人情報がないか
点検して知らせる

点検するのは **participants/taro/ だけ** にする

作り方：Kiro パネル（おばけのアイコン）→ Agent Hooks の「+」→ Ask Kiro to create a hook

STEP 6〜7 Spec：決めてから作らせる

SDD とは　仕様駆動開発（Spec-Driven Development）。AI にいきなり作らせず、
先に「何を・どう作るか」を文書にして人が確認し、その文書どおりに作らせる進め方

いつものチャット　頼む → AI がいきなり作る → できあがり

⚠ 完成してから、
イメージとのずれに気づく

Spec（SDD）　頼む → ① 要件 → ② 設計 → ③ タスク → 実装 → **受け入れ条件で確認**

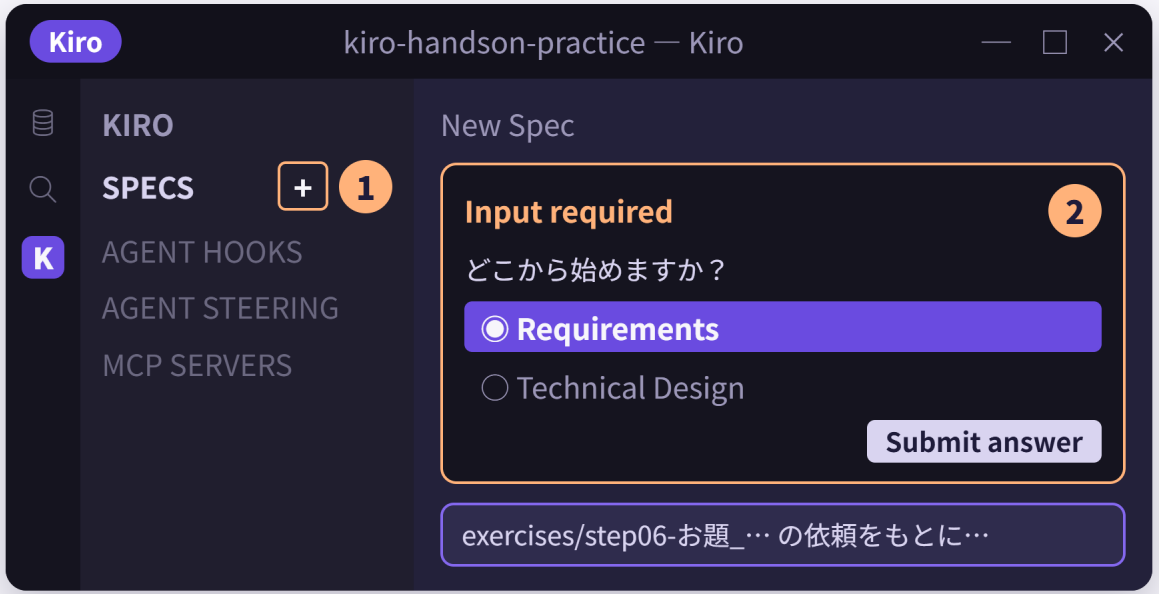
- ① 要件：`requirements.md` — 人が確認
- ② 設計：`design.md` — 人が確認
- ③ タスク：`tasks.md` — 人が確認

✓ 作る前に、文書の段階でずれを直せる。決めた内容も文書で残る

STEP 6：① 要件まで作って PR

STEP 7：② 設計 → ③ タスク → 実装 → 確認

STEP 6 Kiro Spec を始めて、要件を固める



1 Kiro パネル → SPECS の「+」→ 入力欄に貼る

Kiro のチャットに入力

exercises/step06-お題_変更依頼一覧ビューア.md の依頼をもとに、
ツールの要件を作ってください。
Spec の名前は change-request-viewer-taro にしてください。
受け入れ条件は EARS 記法で書いてください。

EARS 記法＝「〜のとき、システムは〜する」という
合否を判定できる書き方

2 Requirements を選んで Submit answer

3 できたら、AI に質問させて穴を埋める

Kiro のチャットに入力

この requirements.md に足りない観点や、
依頼者に確認すべき点を、質問の形で挙げてください。
それぞれに「おすすめの答え」も付けてください。

決められないことは「未確定事項」に書いておく。設計に進むか聞かれても、STEP 6 は要件までで止める

STEP 6〜7 ひとりでもレビューできる

1	ブラウザ	**PR を出す**	base が自分のリポジトリの main になっているか確認する
2	Kiro	**AI がレビュー役**	依頼者の立場でコメントを5つ出させ、review-comments.md に保存
3	ブラウザ	**自分でも見直す**	Files changed で行コメントを付け、見本とも比べる
4	ブラウザ	**受け入れ条件で確かめる**	作ったツールを開き、受け入れ条件ごとに ○／× を付ける
5	ブラウザ	**自分でマージ**	Merge pull request → Confirm merge

自分の PR は自分で Approve できない。4 の確認が承認の代わり

仲間と進めるなら、招待して Reviewers に指定し合う

STEP 7 Kiro タスクは1つずつ「⚡ Start task」



1. **タスクのすぐ上の小さな文字を押す**
   終わったら結果を見て、次のタスクへ

2. **紫の Run all tasks は押さない**
   一気に進めると、
   どこでおかしくなったか分からない

- 書き込み先が participants/taro/ か確認
- HTML をブラウザで開き、
  サンプル CSV で確かめる

STEP 8 モブ演習：その場で要件を決めきる

モブ演習とは

関係者が1つの画面を囲み、その場で要件を詰める進め方。
ひとりのときは Kiro のチャットを2つ（セッション A・B）開き、AI に役を分けて担当させる

セッション A

要件をまとめる AI

たたき台を作り、決めることを質問してくる

自分

A の質問に答えて
自分で決める

セッション B

依頼者役の AI

依頼者にしか分からないことだけ聞く

「あとで考えます」は禁止

決めるか、「未確定事項（確認先つき）」にするかのどちらか

仲間と集まるときは、1つの画面を映し、操作する人（ドライバー）を15分ごとに交代します

STEP 9 応用 スキルで覚えさせ、MCP でつなぐ

Steering
毎回読む前提やルール

Hooks
決まったときに
自動で動く

スキル
呼んだときだけ使う
作業手順書

MCP
外部のツールや
データとつなぐ

スキルを呼ぶ（チャット）

Kiro のチャットに入力

```
/minutes-summary
#materials/議事録_サンプル_変更依頼定例.md
```

MCP をつなぐ（設定ファイル mcp.json に追記）

```
"aws-knowledge": {
  "url": "https://knowledge-mcp.global.api.aws",
  "type": "http" }
```

MCP に送った質問は外部のサービスに届く。個人情報・秘密情報は送らない

STEP 10 上級

AI-DLC：AI が進行役になる

AI-DLC とは

AI 駆動開発ライフサイクル（AI-Driven Development Life Cycle）。AWS が公開している開発の進め方で、
AI が段階ごとに「質問 → 文書 → 人の承認」を繰り返しながら、構想から構築まで進めてくれる

1つの段階で

AI が質問する → **AI が文書にまとめる** → **人が承認 or 差し戻し** → 次の段階へ

全体の流れ

0 初期化 | **1 構想** | **2 方向づけ** | **3 構築** | 4 運用

紫＝この回（poc：試作の型）で通るフェーズ

	Spec（STEP 6〜7）	AI-DLC（STEP 10）
進行役	人が「次へ」を指示する	AI が質問し、承認を求めてくる
残るもの	要件・設計・タスクの3つの文書	段階ごとの文書と、質問・承認の記録
向いている仕事	小さな機能・ツール	関係者が多い・記録を残したい開発

STEP 10 上級 AI-DLC を入れて、試作を1本通す

1 入れて、Kiro 用に設定する

PowerShell Windows PowerShell

```
PS> aidlc version
PS> aidlc config --harness kiro-ide
```

インストール中は Kiro を閉じておく。詳しい手順は手順書を参照

⚠ 型は必ず **poc**（試作）。
指定しないと時間とクレジットを大きく使う

2 Kiro で始める

Kiro のチャットに入力

```
/aidlc poc --learnings off
./vision.md と ./technical-environment.md を読んで…
```

エージェントは aidlc、モデルは Claude Opus を選ぶ

有料プランが必要。
終わったらモデルを Auto に戻す

困ったときの3ステップ

1

困ったとき集を探す

`docs/troubleshooting.md`

2

Kiro に聞く

画面のメッセージを貼って
「どういう意味？」

3

公式ドキュメント

英語のメッセージで
Kiro・GitHub の公式を検索

用語集 docs/glossary.md

プロンプト集 docs/prompts.md

さあ、STEP 0 から

① リポジトリを開いて「Use this template」

② docs/00-environment.md の準備をそろえる

③ docs/self-study.md を読んで STEP 1 へ

`github.com/multilayer/kiro-handson-practice`

画面表記・料金・モデル名は 2026年10月時点。内容は CC BY 4.0 で公開しています